

Sprint 1 Learning Project Guide

Currency Exchange Web App — Django + Keycloak + PostgreSQL + Nginx + Docker + GitFlow

Audience: junior developers with strong computer science fundamentals who are new to this stack.

Goal: build and learn how to ship a small but realistic web application for a fictional currency exchange business.

Sprint 1 Learning Project Guide

Currency Exchange Web App

Django + Keycloak + PostgreSQL + Nginx + Docker + GitFlow

Audience: junior developers with strong computer science fundamentals who are new to this stack.

Goal: build and learn how to ship a small but realistic web application for a fictional currency exchange business.

1. Project goal for Sprint 1

By the end of Sprint 1, the team should have a working application with:

- a Django web app
- Keycloak-based authentication and user management
- PostgreSQL for persistence
- Nginx as the reverse proxy
- Docker Compose to run everything together on Debian
- GitFlow development using `main` and `development`
- a very basic public landing page
- a protected dashboard
- user registration, login, logout, and account deletion

Sprint 1 is intentionally small. Do not add exchange-rate calculations, payment systems, or advanced workflows yet.

2. Recommended sprint architecture

Use the following services:

- **Django**: application logic and server-rendered pages
- **Keycloak**: identity provider and user account management

- **PostgreSQL**: database for Django and Keycloak
- **Nginx**: reverse proxy for browser traffic
- **Docker Compose**: local and Debian deployment

High-level flow

1. User opens the website in the browser.
2. Nginx receives the request.
3. Nginx forwards it to Django or Keycloak.
4. Django sends the user to Keycloak for login or registration.
5. Keycloak authenticates the user.
6. Keycloak sends the user back to Django.
7. Django displays a basic dashboard and account options.

Part A — Jira tasks for Sprint 1

Use one Jira ticket per deliverable. Keep every ticket small enough that it can be completed in a single branch and reviewed independently.

A.1 Suggested epics

- **EPIC-01: Repository and Workflow Setup**
- **EPIC-02: Application Skeleton**
- **EPIC-03: Authentication and User Management**
- **EPIC-04: Infrastructure and Deployment**
- **EPIC-05: Frontend and Sprint Review**

A.2 Sprint 1 task list

FX-01 — Create repository, branches, and protections

Goal: establish the project repo and GitFlow rules.

Acceptance criteria:

- repository exists on GitHub
- `main` and `development` branches exist
- branch protections are enabled
- no one pushes directly to `main` or `development`
- PRs are required for merges

Suggested subtasks:

- create repo
- create `development`
- protect both branches
- add team access

FX-02 — Add Docker Compose skeleton

Goal: create the folder layout and initial Compose file.

Acceptance criteria:

- `docker-compose.yml` exists
- folders for Django, Nginx, and Keycloak config exist
- `docker compose up` starts placeholder services without errors

FX-03 — Bootstrap Django project

Goal: create the first Django project and app.

Acceptance criteria:

- Django project starts in a container
- the homepage returns a basic response
- static and templates folders exist

FX-04 — Add PostgreSQL connectivity

Goal: connect Django to PostgreSQL.

Acceptance criteria:

- PostgreSQL container starts
- Django connects to the database
- migrations run successfully
- database credentials come from environment variables

FX-05 — Set up Keycloak realm and client

Goal: create the identity provider configuration.

Acceptance criteria:

- Keycloak container starts
- a project realm exists
- a client for Django exists
- redirect URIs are configured
- user registration is enabled

FX-06 — Integrate Django login through Keycloak

Goal: make browser login work through Keycloak.

Acceptance criteria:

- user can click login from Django
- user is redirected to Keycloak
- successful login returns to Django
- Django displays authenticated user information

FX-07 — Add registration and account deletion flow

****Goal:**** let users create and remove their own accounts.

****Acceptance criteria:****

- registration starts from the browser
- account creation is handled by Keycloak
- account deletion is possible from a browser page
- deleted user cannot log in again

FX-08 — Add Nginx reverse proxy

****Goal:**** place Nginx in front of Django and Keycloak.

****Acceptance criteria:****

- browser traffic enters through Nginx
- Nginx forwards requests to Django and Keycloak
- static assets can be served or proxied correctly

FX-09 — Build a basic branded frontend

****Goal:**** create a simple currency exchange themed UI.

****Acceptance criteria:****

- landing page has business branding
- dashboard shows user status
- layout is readable and simple
- login/logout/delete buttons are visible where appropriate

FX-10 — Write smoke tests and manual checklist

****Goal:**** provide a repeatable validation process.

****Acceptance criteria:****

- checklist covers startup, login, registration, deletion, logout
- team can validate the stack after each merge

FX-11 — Write deployment and troubleshooting guide

****Goal:**** document the system for new developers.

****Acceptance criteria:****

- guide explains local startup
- guide explains how to reset the environment
- guide includes the most common failure points

A.3 Suggested Jira order

Recommended order for Sprint 1:

1. FX-01
2. FX-02
3. FX-03
4. FX-04
5. FX-05
6. FX-06
7. FX-08
8. FX-09
9. FX-07
10. FX-10
11. FX-11

The reason for this order is dependency flow: build the repo and runtime first, then application and identity, then proxy and frontend, then tests and documentation.

Part B — GitHub GitFlow integration and branch management

B.1 Branch model

Use these branches:

- ``main`` — stable and release-ready
- ``development`` — integration branch for sprint work
- ``feature/<jira-key>-short-name`` — one branch per task
- ``release/<version>`` — optional stabilization branch near sprint end
- ``hotfix/<version>`` — only for urgent fixes on ``main``

Example branches

- ``feature/FX-03-django-bootstrap``
- ``feature/FX-06-keycloak-login``
- ``feature/FX-09-basic-frontend``

B.2 GitFlow rules for this project

1. Never commit directly to ``main``.
2. Never commit directly to ``development``.
3. Every Jira ticket must use its own feature branch.
4. Every branch must go through a PR.
5. Use merge commits so branch history remains visible.
6. Keep merged branches visible until the sprint review is complete.
7. Protect ``main`` and ``development`` with branch rules.

B.3 Suggested GitHub branch protections

For both `main` and `development`, enable:

- pull request required
- at least one review required
- status checks required if available
- no force pushes
- no direct pushes

If the team wants history to remain easy to inspect, avoid squash merging.

B.4 Step-by-step GitFlow workflow

Step 1 — Clone the repository

```
git clone git@github.com:YOUR-ORG/YOUR-REPO.git
cd YOUR-REPO
```

Step 2 — Make sure you are on `development`

```
git checkout development
git pull origin development
```

Step 3 — Create a feature branch for one Jira ticket

```
git checkout -b feature/FX-06-keycloak-login
```

Step 4 — Work only on that task

Keep changes small and relevant to the ticket.

Good commit style:

```
git add .
git commit -m "FX-06: add Keycloak login callback"
```

Step 5 — Push the branch

```
git push -u origin feature/FX-06-keycloak-login
```

Step 6 — Open a pull request

Target branch:

- source: `feature/FX-06-keycloak-login`
- target: `development`

Include in the PR description:

- Jira ticket number
- what changed
- how it was tested
- any known limitations

Step 7 — Review the PR

Reviewer checklist:

- ticket scope is respected
- code follows the current project pattern

- tests or manual checks were run
- no secrets were committed

Step 8 — Merge the PR

Use a ****merge commit**** to preserve visible branch history.

Step 9 — Leave the branch visible for review

Do not delete the branch immediately if the team wants it visible during sprint review.

Step 10 — Repeat for the next ticket

Always branch from the latest `development`.

B.5 Using GitHub and VSCode together

In VSCode

1. Open the repo folder.
2. Open the Source Control panel.
3. Pull the latest `development`.
4. Create a new branch from the branch picker.
5. Make edits.
6. Stage files in Source Control.
7. Commit with a clear message.
8. Push from VSCode.
9. Create the PR on GitHub or from the GitHub Pull Requests extension.

Useful VSCode habits

- use the branch indicator in the bottom-left corner
- keep the Git graph visible if possible
- resolve merge conflicts in the editor, not in the terminal when you are learning
- prefer small PRs

B.6 Simple Git commands the juniors should know

```
git status
git branch
git checkout development
git checkout -b feature/FX-09-basic-frontend
git add .
git commit -m "FX-09: add landing page layout"
git push -u origin feature/FX-09-basic-frontend
git pull origin development
```

If they need to inspect history:

```
git log --oneline --graph --decorate --all
```

Part C — Step-by-step Django, Keycloak, PostgreSQL, Nginx, and Docker setup on Debian

C.1 Install prerequisites on Debian

On the Debian machine, install the base tools first.

```
sudo apt update
sudo apt install -y git ca-certificates curl
```

Install Docker Engine and the Docker Compose plugin using your standard Debian installation method.

After installation, verify:

```
docker --version
docker compose version
```

If needed, add your user to the docker group:

```
sudo usermod -aG docker $USER
newgrp docker
```

C.2 Create the project structure

A simple directory layout:

```
project/
├── backend/
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/
│   └── app/
├── nginx/
│   └── default.conf
├── keycloak/
│   └── realm-export.json
├── docker-compose.yml
├── .env
└── README.md
```

Create it with:

```
mkdir -p project/backend project/nginx project/keycloak
cd project
```

C.3 Create the Django project

Inside `backend`, create a virtual environment if you want local development outside Docker, or build everything into Docker from the beginning.

For a Django start inside Docker, the usual learning flow is:

1. create `requirements.txt`
2. create a `Dockerfile`
3. create the project inside the container

Example `requirements.txt`:

```
Django==5.1.0
gunicorn==22.0.0
psycpg2-binary==2.9.9
```



```
python-dotenv==1.0.1
mozilla-django-oidc==4.0.1
```

If your team uses a different OIDC package, keep the setup consistent across all developers.

C.4 Basic Django Dockerfile

Example `backend/Dockerfile`:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app/

CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

C.5 Create Docker Compose services

Example `docker-compose.yml` outline:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: exchange_db
      POSTGRES_USER: exchange_user
      POSTGRES_PASSWORD: exchange_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  keycloak-db:
    image: postgres:16
    environment:
      POSTGRES_DB: keycloak_db
      POSTGRES_USER: keycloak_user
      POSTGRES_PASSWORD: keycloak_password
    volumes:
      - keycloak_postgres_data:/var/lib/postgresql/data

  keycloak:
    image: quay.io/keycloak/keycloak:latest
    command: start-dev
    environment:
      KC_DB: postgres
      KC_DB_URL_HOST: keycloak-db
      KC_DB_URL_DATABASE: keycloak_db
      KC_DB_USERNAME: keycloak_user
      KC_DB_PASSWORD: keycloak_password
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin_password
```

```

depends_on:
  - keycloak-db
ports:
  - "8080:8080"

web:
  build: ./backend
  command: gunicorn config.wsgi:application --bind 0.0.0.0:8000
  environment:
    DATABASE_URL: postgres://exchange_user:exchange_password@db:5432/exchange_db
  depends_on:
    - db

nginx:
  image: nginx:stable
  ports:
    - "80:80"
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    - web
    - keycloak

volumes:
  postgres_data:
  keycloak_postgres_data:

```

This is a starting point. Your exact Keycloak image and settings may vary.

C.6 Start PostgreSQL and verify it works

Bring up only the database first:

```
docker compose up -d db
```

Check the logs:

```
docker compose logs -f db
```

If the database starts correctly, Django should be able to connect to it.

C.7 Configure Django database settings

In Django settings, read database values from environment variables.

Example idea:

```

import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("POSTGRES_DB", "exchange_db"),
        "USER": os.getenv("POSTGRES_USER", "exchange_user"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD", "exchange_password"),
        "HOST": os.getenv("POSTGRES_HOST", "db"),
        "PORT": os.getenv("POSTGRES_PORT", "5432"),
    }
}

```

Run migrations inside the container:

```
docker compose run --rm web python manage.py migrate
```

If you need an admin user for Django admin debugging:

```
docker compose run --rm web python manage.py createsuperuser
```

C.8 Create the Django app and first page

Create an application for the business pages:

```
docker compose run --rm web python manage.py startapp core
```

Then add a simple homepage view and template.

Example `core/views.py`:

```
from django.shortcuts import render

def home(request):
    return render(request, "core/home.html")
```

Add a URL route for the homepage.

Your first page should confirm that Django is running before you add authentication.

C.9 Set up Keycloak

Start Keycloak:

```
docker compose up -d keycloak keycloak-db
```

Open Keycloak in the browser:

- `http://localhost:8080`

Log in as the admin user you configured in Compose.

In the Keycloak admin console

1. Create a new realm, for example `exchange-learning`.
2. Create a client for Django.
3. Set the client type according to your chosen OIDC flow.
4. Enable the authorization code flow.
5. Add valid redirect URIs for Django.
6. Add web origins if required.
7. Enable registration if the sprint requires self-service account creation.
8. Create test roles such as `customer`.

C.10 Integrate Django with Keycloak using OIDC

For Sprint 1, the simplest approach is to let Keycloak handle identity and let Django consume the login result.

Typical steps:

1. Install an OIDC library in Django.
2. Configure the Keycloak issuer URL.
3. Configure the client ID and client secret.
4. Define login and logout routes.
5. Define the callback URL.

6. Store the authenticated user in a Django session.

Example environment values:

```
KEYCLOAK_SERVER_URL=http://keycloak:8080/  
KEYCLOAK_REALM=exchange-learning  
KEYCLOAK_CLIENT_ID=exchange-web  
KEYCLOAK_CLIENT_SECRET=change-me
```

Example Django auth flow:

- user clicks `Login`
- Django redirects the browser to Keycloak
- Keycloak authenticates the user
- Keycloak redirects back to Django
- Django creates or updates the session

C.11 Configure logout

Logout should do two things:

1. end the Django session
2. end the Keycloak session or redirect through the Keycloak logout endpoint

After logout, the user should return to the public landing page.

C.12 Configure account deletion

For Sprint 1, account deletion should be handled by Keycloak or through a Keycloak-backed account management flow.

Recommended user experience:

1. user opens their account page
2. user clicks `Delete account`
3. application shows a confirmation page
4. the delete action is sent to the Keycloak account lifecycle path
5. session is cleared
6. the browser returns to the landing page

Important: require confirmation before deletion.

C.13 Add Nginx reverse proxy

Nginx should be the browser-facing entry point.

Example `nginx/default.conf`:

```
server {  
    listen 80;  
    server_name _;  
  
    location / {  
        proxy_pass http://web:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

If you also proxy Keycloak through the same Nginx instance, add a separate location block or a separate hostname.

C.14 Start the full stack

When the containers and config are ready:

```
docker compose up -d --build
```

Check service status:

```
docker compose ps
```

Read logs when something fails:

```
docker compose logs -f web
docker compose logs -f keycloak
docker compose logs -f nginx
```

C.15 Useful restart and cleanup commands

Stop the stack:

```
docker compose down
```

Stop and remove volumes if you want a clean reset:

```
docker compose down -v
```

Rebuild after code changes:

```
docker compose up -d --build
```

Part D — Frontend design and user management

D.1 Design philosophy for Sprint 1

The frontend should be simple, clear, and believable for a currency exchange business.

The goal is not visual sophistication. The goal is to make the user flow obvious:

- public visitor sees the company
- user logs in or registers
- user lands on a dashboard
- user can log out or delete the account

D.2 Suggested pages

Public pages

- landing page
- login page redirect button

- registration button

Authenticated pages

- dashboard
- account page
- delete account confirmation page

D.3 Suggested page content

Landing page

Include:

- business name
- short description of currency exchange services
- login button
- register button

Example text:

> Fast and transparent currency exchange for global customers.

Dashboard

Include:

- logged-in username
- email address if available
- user role if available
- logout button
- delete account button

Account page

Include:

- profile summary
- warning before deletion
- confirm delete button

D.4 Simple visual style

Use a restrained palette:

- navy blue
- teal or green accent
- white background
- gray text

Keep typography readable and spacing generous.

Recommended design patterns:

- top navigation bar

- card-based sections
- one primary button per action
- minimal icons

D.5 How to build the frontend in Django

Use Django templates rather than a heavy frontend framework.

Step 1 — Create a base template

`base.html` should contain:

- site header
- navigation
- content block
- footer

Step 2 — Create the landing page template

Show the product summary and login/register buttons.

Step 3 — Create the dashboard template

Show user details and session state.

Step 4 — Create the account template

Show account options and delete confirmation.

D.6 User management behavior

Login

1. user clicks login
2. browser goes to Keycloak
3. successful authentication returns to the app
4. dashboard loads

Register

1. user clicks register
2. browser goes to Keycloak registration
3. new account is created
4. user returns to the app

Logout

1. user clicks logout
2. Django session ends
3. Keycloak session ends or is invalidated
4. user returns to public page

Delete account

1. user opens account page
2. user clicks delete
3. confirmation is required
4. Keycloak account is deleted
5. session is cleared
6. user cannot log in again with that account

D.7 Definition of done for the frontend

The Sprint 1 frontend is done when:

- the public page looks like a currency exchange business
- the user can click through to Keycloak login
- the user can register an account
- the user can log in
- the user can log out
- the user can delete the account
- the authenticated view shows a simple dashboard

Part E — Suggested implementation order for juniors

Follow this order if the team wants the least confusion:

1. create repo and GitFlow branches
2. create Docker Compose skeleton
3. add PostgreSQL
4. bootstrap Django
5. create first homepage
6. create Keycloak realm and client
7. connect Django login to Keycloak
8. add logout
9. add account deletion
10. add Nginx reverse proxy
11. improve the frontend layout
12. write smoke tests and documentation

Part F — Practical commands cheat sheet

Git

```
git checkout development
git pull origin development
git checkout -b feature/FX-06-keycloak-login
```



```
git status
git add .
git commit -m "FX-06: add Keycloak login flow"
git push -u origin feature/FX-06-keycloak-login
```

Docker

```
docker compose up -d --build
docker compose ps
docker compose logs -f web
docker compose down
docker compose down -v
```

Django

```
docker compose run --rm web python manage.py migrate
docker compose run --rm web python manage.py createsuperuser
docker compose run --rm web python manage.py startapp core
```

Keycloak

- open `http://localhost:8080`
- create realm
- create client
- configure redirect URIs
- enable registration

Part G — Sprint 1 success criteria

Sprint 1 is successful if all of the following are true:

- the application can be started with Docker Compose
- Django uses PostgreSQL
- Keycloak manages user login and registration
- the browser can create, log in, and delete an account
- Nginx is the reverse proxy entry point
- the frontend is simple but clearly branded for a currency exchange business
- each Jira ticket was merged separately through GitFlow
- merged branches remain visible for sprint review

Part H — Common beginner mistakes

Mistake 1: building too much UI too early

Keep the first version minimal.

Mistake 2: storing passwords in Django

Do not do that. Use Keycloak for identity.

Mistake 3: mixing multiple tickets in one branch

One ticket, one branch, one PR.

Mistake 4: skipping the database verification step

Always confirm PostgreSQL works before debugging authentication.

Mistake 5: making Nginx too complex

Start with one proxy path, then expand.

Mistake 6: deleting merged branches too early

Keep them visible until the team finishes review.

Part I — Final checklist

Before calling Sprint 1 complete, verify:

- ☐ repo and branches are ready
- ☐ Docker Compose starts the stack
- ☐ Django can connect to PostgreSQL
- ☐ Keycloak realm and client are configured
- ☐ login works in the browser
- ☐ registration works
- ☐ logout works
- ☐ account deletion works
- ☐ Nginx proxies browser traffic
- ☐ landing page is branded
- ☐ dashboard is visible after login
- ☐ Jira tickets are merged separately
- ☐ documentation is complete

End of guide